

Decorator:

Python decorator is function that adds functionality to another function without changing structure of original function.

In [1]:

```
def inc(x):  
    return (x+1)  
def operate (func, x):  
    result = func(x)  
    return result  
print(operate(inc,3))  
  
#function can take an another function as an argument
```

4

In [4]:

```
##Function inside a function  
  
def print_msg(message):  
    greeting = "Hello"  
  
    def printer():  
        print(greeting,message)  
    printer()  
print_msg("Python is Awsome")  
  
# func = print_msg("Python is Awesome")  
# func()
```

Hello Python is Awsome

In [52]:

```
## Function with Parameters

def smart_divide(func):
    def inner(a, b):
        print("I am going to divide", a, "and", b)
        if b == 0:
            print("Whoops! cannot divide")
            return

        return func(a, b)
    return inner

@smart_divide
def divide(a, b):
    print(a/b)

value_1 = divide(15,3)
print(value_1)

value_2 = divide(5,0)
print(value_2)
```

```
I am going to divide 15 and 3
5.0
None
I am going to divide 5 and 0
Whoops! cannot divide
None
```

In [44]:

```
def display_info(func):
    def inner():
        print("Executing function")
        func()
        print("Finished Execution")
    return inner

@display_info
def printer():
    print("Hello World!")

printer()
```

```
Executing function
Hello World!
Finished Execution
```

In [40]:

```
def printer():
    print("Hello World!")

def dispaly_info(func):
    def inner():
        print("Executing", func.__name__, "function")
        func()
        print("Finished Execution")
    return inner
printer()
```

Hello World!

In [51]:

```
def lowercase_decorator(function):
    def wrapper():
        func = function()
        string_lowercase = func.lower()
        return string_lowercase
    return wrapper
@lowercase_decorator
def lower():
    return 'HELLO WORLD'
lower()
```

Out[51]:

'hello world'

One-to-One Field:

This is used when one record of a model A is related to exactly one record of another model B. This field can be useful as a primary key of an object if that object extends another object in some way.

example – a model Car has one-to-one relationship with a model Vehicle, i.e. a car is a vehicle. One-to-one relations are defined using OneToOneField field of django.db.models.

In [55]:

```
from django.db import models

class Vehicle(models.Model):
    reg_no = models.IntegerField()
    owner = models.CharField(max_length = 100)

class Car(models.Model):
    vehicle = models.OneToOneField(Vehicle,
        on_delete = models.CASCADE, primary_key = True)
    car_model = models.CharField(max_length = 100)
```

Many-to-many fields:

This is used when one record of a model A is related to multiple records of another model B and vice versa.

example – a model Book has many-to-many relationship with a model Author, i.e. an book can be written by multiple authors and an author can write multiple books. Many-to-many relations are defined using ManyToManyField field of django.db.models.

In []:

```
from django.db import models

class Author(models.Model):
    name = models.CharField(max_length = 100)
    desc = models.TextField(max_length = 300)

class Book(models.Model):
    title = models.CharField(max_length = 100)
    desc = models.TextField(max_length = 300)
    authors = models.ManyToManyField(Author)
```

Many-to-one field (ForeignKey):

This is used when one record of a model A is related to multiple records of another model B.

example – a model Song has many-to-one relationship with a model Album, i.e. an album can have many songs, but one song cannot be part of multiple albums. Many-to-one relations are defined using ForeignKey field of django.db.models.

In []:

```
from django.db import models

class Album(models.Model):
    title = models.CharField(max_length = 100)
    artist = models.CharField(max_length = 100)

class Song(models.Model):
    title = models.CharField(max_length = 100)
    album = models.ForeignKey(Album, on_delete = models.CASCADE)
```

In []:

In []:

In []: